

| NO. | 토픽   | 중요도 | 중토픽               | 키워드 특징 및 설명                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | 타연관 토픽                                                            | 암기법 |
|-----|------|-----|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|-----|
| 1   | 알고리즘 | 중   | O-Notation        | [정의] 데이터 두 N에 대해서 복잡도가 어떤 함수로 나타나는가를 간단히 표현하기 위한 알고리즘 표현법<br>[복잡도] 시간적복잡도 - 시간 효율성을 위해 알고리즘 수행에 필요한 시간의 양측정<br>점근적인 복잡도 - 매개변수 N의 값이 무한에 가까워졌을 때 복잡도가 어떻게 되느냐에 중점을 두고 평가<br>[알고리즘 분석 시 고려사항] 정확성, 작업량, 기억장소 사용량, 단순성, 최적성                                                                                                                                                                                                                                                                                    |                                                                   |     |
| 2   |      | 하   | 재귀 알고리즘           | 재귀함수로 구현이 가능한 것들이 Divide&Conquer                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | 알고리즘 설계기법:분할과 정복 사례                                               |     |
| 3   |      | 중   | 알고리즘 평가 방법        | [정의] 작업 알고리즘을 구현하지 않고 수행시간을 분석하는 기법<br>[유형] 시간복잡도 - 얼마나 빠른가?<br>공간복잡도 - 메모리 사용량 분석, 얼마나 적게 쓰는가?<br>[표기법] Big O - O(n), 점근적 상한선, 어떤 최악의 상황에서도 성능보장<br>Big Omega - 점근적 하한선, 아무리 좋은 케이스라도 그 성능 이상 낼 수 없음<br>Big Theta - 점근선 상한선과 하한선의 교집합, 수행시간 상한,하한 선이 표현                                                                                                                                                                                                                                                     | 시간 복잡도(Time Complexity)<br>공간 복잡도(Space Complexity)<br>그림 찾아보기    |     |
| 4   | 정렬   | 상   | 정렬 분류             | 맨 앞 부분에는 개념과 분류를 기술해야 하니 아래 정렬 알고리즘의 분류를 하고, 개념과 키워드는 암기 요망!<br>[정의] 집합 안 두개의 원소를 비교하여 정렬되어 있지 않으면 교환하는 정렬 알고리즘.<br>데이터 집합을 순회하면서 집합내의 이웃요소들끼리 교환통해 정렬<br>[특징] 매우 간단하나 비효율적, 내부정렬(주기억장치에서 수행), 교환법 맨 뒤부터 정렬<br>[소스코드]<br>void bubbleSort( TYPE a[],int n ) {<br>TYPE t; bool sorted;<br>for(int i =0; i < n -1; i++) {<br>sorted =true;<br>for(int j =1; j < n -1; j++) {<br>if( a[j -1]> a[j] ) {<br>t = a[j -1]; a[j -1]= a[j];<br>a[j] = t; sorted =false;<br>}<br>}<br>if( sorted )break; // 정렬되어 있으면 끝냄 |                                                                   |     |
| 5   |      | 하   | 버블정렬              | [수행방법] 정렬되어 있는 부분집합에 정렬할 새로운 원소의 위치를 찾아 삽입<br>데이터 집합을 순회하면서 정렬이 필요한 요소를 뽑아내 이를 다시 적당한 곳에 삽입해 나가는 알고리즘                                                                                                                                                                                                                                                                                                                                                                                                        | 성능 O(n <sup>2</sup> )<br>비교->교환 과정의 반복수행<br>flag 0 or 1           |     |
| 6   |      | 하   | 삽입정렬              | [수행방법] 특정 레코드와 거리가 일정한 레코드끼리 부파일로 나누어 각 부파일을 삽입 정렬하는 방식<br>부파일로 나누기 위해 거리 h를 점점 작게 하여 h가 1이 되면 정렬이 끝남                                                                                                                                                                                                                                                                                                                                                                                                        | 성능 O(n <sup>2</sup> )<br>장점 - 필요한 경우에만 배열 정렬                      |     |
| 7   |      | 하   | 선택정렬              | [수행방법] 특정 레코드와 거리가 일정한 레코드끼리 부파일로 나누어 각 부파일을 삽입 정렬하는 방식<br>부파일로 나누기 위해 거리 h를 점점 작게 하여 h가 1이 되면 정렬이 끝남                                                                                                                                                                                                                                                                                                                                                                                                        | 부분 리스트 반복, 최악 O(N <sup>2</sup> ), 평균 O(N1.5), h개 분할, 원거리          |     |
| 8   |      | 중   | 퀵정렬               | [수행방법] 정렬한 전체 원소에 대해서 정렬을 수행하지 않고, 기준 값을 중심으로 왼쪽, 오른쪽 집합으로 분할하여 정렬하는 방법<br>1)리스트를 두개로 분할 2)좌측 리스트 가장 왼쪽값을 Pivot로 선정 3)Pivot보다 작은값은 왼쪽으로 큰 값들은 오른쪽으로 이동(순서무관) 4)1~3의 과정을 반복->정렬된 데이터 집합                                                                                                                                                                                                                                                                                                                       | pivot, LR, O(nlogn)<br>분할정복(Divide and Conquer)<br>평균 실행속도 가장 뛰어나 |     |
| 9   |      | 하   | 선택정렬(Select sort) | [수행방법] 전체 원소중에서 가장 작은 원소를 찾아서, 첫 번째 원소와 자리 바꿈<br>[소스코드]<br>void selection_sort(int arr[], int size) {<br>int i,j,k,t;<br>for(i=0;i<size;i++) {<br>k = i;<br>for(j=i+1;j<size;j++) { if(arr[k]>arr[j]) { k=j; } }<br>t = arr[k]; arr[k]=arr[i]; arr[i]=t;<br>}                                                                                                                                                                                                                                                | 전체 시간복잡도 : O(n <sup>2</sup> )                                     |     |
| 10  |      | 중   | 병합정렬(Merge Sort)  | [정의] 하나의 리스트를 기준값으로 분할하고, 정렬하면서 정렬하는 알고리즘<br>[원리]1)미정렬 리스트를 두개의 서브리스트로 분할 2)각 서브리스트 크기가 1이 될때까지 recursive하게 분할 3)분리된 서브리스트를 정렬하면서 병합<br>정렬의 대상이 되는 자료를 부분 구조로 나눈 후, 재귀적으로 병합하여 정렬하는 정렬 알고리즘<br>O(n(log n))의 정렬 알고리즘이며 Divide-and-Conquer 정책을 따름<br>- 퀵 정렬에서는 분할되는 두 부분배열의 크기가 다를 수 있어, 최악의 성능이 O(n <sup>2</sup> )으로 될 수있음                                                                                                                                                                                    | divide-conquer, O(nlogn), 재귀함수                                    |     |
| 11  |      | 중   | 힙정렬               | [정의] 완전이진트리에 있는 노드 중에서 키 값이 가장 큰 노드나 키 값이 가장 작은 노드를 찾기 위해 만든 자료구조<br>[시간복잡도] O(N log N)<br>[공간복잡도] O(1)                                                                                                                                                                                                                                                                                                                                                                                                      | 완전이진트리<br>오름차순 : 최대힙(Max Heap)<br>내림차순 : 최소힙(Min Heap)            |     |
| 12  |      | 하   | 기수정렬(Radix Sort)  | 레코드를 비교하지 않고 준비된 버킷을 이용하여 정렬하는 알고리즘<br>특징 정렬레코드 타입제한, 시간복잡도: o(KN) ; k 버킷의 수                                                                                                                                                                                                                                                                                                                                                                                                                                 |                                                                   |     |
| 13  |      | 하   | 계수정렬              | [정의] 항목들의 순서를 결정하기 위해 집합에 각 항목이 몇 개씩 있는지 세는 작업을 하면서<br>선형 시간에 정렬하는 효율적인 알고리즘<br>[특징] Stable Sort, 중복 데이터 정렬, 비교정렬 아님                                                                                                                                                                                                                                                                                                                                                                                         |                                                                   |     |
| 14  |      | 중   | 트리정렬(Tree Sort)   | [정의] 이진 탐색 트리 형태의 구조를 띄면서 데이터가 삽입/삭제될 경우 트리탐색 과정을 거쳐 재구성한 후, 키 값을 삽입 후 중위 순회를 수행하는 정렬 알고리즘                                                                                                                                                                                                                                                                                                                                                                                                                   |                                                                   |     |
| 15  |      | 하   | 외부정렬              | 정렬 할 데이터의 크기가 주기억장소 용량보다 클 경우 외부 기억장치(메모리 외부), 디스크 장치파일을 직접 또는 순차적으로 접근하여 정렬하는 알고리즘                                                                                                                                                                                                                                                                                                                                                                                                                          | 디스크                                                               |     |

|    |          |      |                                  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |                                                                                                                           |                       |
|----|----------|------|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|-----------------------|
| 16 | 해시(Hash) | 상    | 해시테이블                            | <p>[정의] 해시 함수에 의해 산출된 주소 값의 위치에 각 레코드를 기억 시킨 테이블</p> <p>[장점]</p> <ul style="list-style-type: none"> <li>-레코드를 찾기 위하여 키 값을 모두 비교할 필요 없음 -키 값을 직접 주소로 사용하는 경우보다 기억장소의 낭비를 줄일 수 있음</li> <li>-레코드가 저장되어 있는 주소를 직접 계산에 의해 얻을 수 있으므로 탐색속도가 빠름</li> <li>-오버플로우 또는 충돌이 발생하지 않으면 원하는 레코드에 한번에 접근할 수 있음</li> </ul> <p>[단점]</p> <ul style="list-style-type: none"> <li>-모든 키 값을 숫자(정수형)로 변환해야함.</li> <li>-키의 변환을 통하여 생성되는 주소가 주소 공간에 적절히 분포 될 수 있도록 하는 해시 함수를 구하는 [문제가 존재]</li> <li>-해시 테이블을 위한 기억장소의 할당을 위해 버킷의 개수와 크기를 적절히 결정 필요</li> </ul> <p>[충돌해결(오버플로우)의 여러가지 기법]</p> <ul style="list-style-type: none"> <li>-개방주소법 : 선 재 이 무 (선형조사법, 재해싱=이중해싱, 이차조사법, 무작위검색)</li> <li>-폐쇄주소법: 체(Chaining: coalesced chaining ;합병 체이닝, separate chaining;분리 체이닝)</li> </ul> <p>[활용분야]</p> <p>해시탐색, 인덱스; 레코드의 위치 주소값</p> | 해시탐색, O(1)<br>오버플로우 해결기법: 선재이/체오(만원)                                                                                      |                       |
| 17 |          | 상    | 해시함수(알고리즘 측면)                    | <p>[정의] 임의의 길이에 이진 문자열을 고정된 길이의 이진 문자열(해쉬값, 메시지 다이제스트, 메시지 지문)으로 매핑하는 함수</p> <p>[특징] 역상저항성(one way function), 약한 충돌 회피, 강한 충돌 회피 (필수 조건) 압축성(임의 길이로 출력), 효율성(빠른 계산), 단방향성(one wayness), 충돌 회피성(collision resistance)</p> <p>[종류] 나눗셈법, 폴딩법, 중간제곱법, 기수변환, 자릿수분석, 무작위법 (충돌해결방법)</p> <p>(1) 개방주소 : 선형조사, 이중해싱, 이차조사, 무작위<br/>(2) 폐쇄주소 : 해시체이닝, 콜라세싱 해싱<br/>[해싱종류] 완전해싱, 정적해싱, 동적해싱</p>                                                                                                                                                                                                                                                                                                                                                                                                                |                                                                                                                           | 압효단충 나뉘리는 중세기 선충이무 체코 |
| 18 |          | 하    | 그래프                              | <p>[정의] 연결된 객체 간의 정점과 간선으로 표현하는 자료구조</p> <p>[표기법]</p> <ol style="list-style-type: none"> <li>1. 무방향 그래프 : <math>(A, B) = (B, A)</math></li> <li>2. 방향 그래프 : <math>\langle A, B \rangle \neq \langle B, A \rangle</math></li> <li>3. 가중치 그래프 : <math>A \rightsquigarrow 40 \rightsquigarrow B</math></li> </ol> <p>[구성요소] 차수(Degree), 허브(Hub), 워크(Walk), 경로(Path), 지름(Diameter), 반지름(Radius), 중심성(Centrality), 밀도(Density), 그래프 엔트로피(Graph Entropy)</p> <p>[표현방법] 이점행렬, 인접리스트, 가선리스트</p> <p>한 노드에서 네트워크 다른 모든 노드에 이르는 최소 비용(least-cost) 경로 계산</p> <p>[계산 복잡도] 각 반복마다 모든 노드 검사 <math>O(n^2)</math>, 효과적구현시 <math>O(n \log n)</math><br/>[전통문제] 노드 경로비용 유사 시 근사비용간 전통문제 발생 Prim 알고리즘, 거리벡터 알고리즘 사용</p> <p>[특징] Greedy method, 선분해, shortest path</p>                                                  |                                                                                                                           | 무방가                   |
| 19 |          | 상    | Dijkstra 알고리즘                    | <p>유리식 접근으로 출발노드에서 목표노드까지 경로가 존재하는 모든 그래프에 대해 목표노드까지 최적경로를 항상 유도하는 알고리즘</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | 계진<br>균일비용알고리즘                                                                                                            | 123기출                 |
| 20 |          | 중    | A* 알고리즘                          | <p>휴리스틱 접근으로 출발노드에서 목표노드까지 경로가 존재하는 모든 그래프에 대해 목표노드까지 최적경로를 항상 유도하는 알고리즘</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |                                                                                                                           | 123기출                 |
| 21 | 그래프      | 중    | 벨만포드 알고리즘                        | <p>음수 가중치가 있고, 음수 사이클이 없는 그래프에서 한 정점으로부터 다른 모든 정점으로의 최단 경로 및 거리를 구하는 알고리즘</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |                                                                                                                           | 123기출                 |
| 22 |          | 중    | 플로이드와샬 알고리즘                      | <p>모든 정점 사이의 최단경로 및 최단 거리를 구하는 알고리즘</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |                                                                                                                           | 123기출                 |
| 23 |          | 하    | 8-퍼즐                             | <p>A* 알고리즘 적용, 평가함수</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |                                                                                                                           |                       |
| 24 |          | 출제예상 | 그래프 순회                           | <p>그래프의 모든 정점(vertex)를 체계적으로 방문하는 것</p> <p>깊이 우선 탐색(DFS: depth first search) : 스택 사용<br/>너비 우선 탐색(BFS: breadth first search) : 큐 사용</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | DFS: depth first search<br>BFS: breadth first search                                                                      |                       |
| 25 |          | 상    | 최소 신장 트리(MST)                    | <p>[정의] 방향성 그래프에서 정점을 연결하는 간선의 가중치 합이 최소가 되는 신장트리</p> <p>[구현알고리즘] 크루스칼, 프림, SOLIN</p> <p>[특징] 그래프 내 원래 있던 edge들만 포함, cycle(순환)없음, vertex수가 n일때 n-1개 edge만 포함</p> <p>[비용인접리스트] 각 정점의 노드와 노드의 가중치 표기한 그래프 표현 방식</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | 신장트리 : 그래프에서 모든 정점을 포함하고, 정점간 서로 연결되면서 사이클이 존재하지 않는 그래프<br>활용: 네트워크설계(라우터 경로 설정, 최적의 통신경로 선택)<br>수송시스템(도시 계획시 고속도로 건설 계획) |                       |
| 26 |          | 중    | 최단 경로 탐색                         | <p>Dijkstra 알고리즘, A* 알고리즘</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |                                                                                                                           |                       |
| 27 |          | 출제예상 | DAG(Directed Acyclic Graph) 알고리즘 | <p>방향성 비순환 그래프에서 개별 요소들은 블록체인처럼 여러 개의 트랜잭션을 하나의 블록으로 묶지 않고, 각 개별 요소들끼리 상호 연결되어 있다. 방향성 비순환 그래프는 시간적으로 이전 트랜잭션에 대해 그 이후 트랜잭션이 검증하는 구조</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | 위상 정렬                                                                                                                     |                       |
| 28 |          | 중    | Huffman                          | <p>[정의] 가변 길이 부호화로서 인도가 높은 데이터는 짧은 부호를, 낮은 데이터는 긴 부호를 할당하여 전체 데이터의 길이를 원본 데이터의 길이보다 짧게 하는 압축 알고리즘</p> <p>[활용] 데이터 압축, 멀티미디어 압축</p> <p>[절차] 1. 모든 문자들의 빈도수를 참고하여 내림차순으로 정렬<br/>2. 가장 적은 빈도수 두 문자를 연결하고 다시 이 연결된 문자들과 다음으로 적은 것과 연결. 모든 문자가 연결될 때까지 반복 (Binary Fusion)</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |                                                                                                                           |                       |
| 29 |          | 중    | Run-Length Coding                | <p>비손실 압축, 반복문자와 탈출문자</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |                                                                                                                           |                       |
| 30 |          | 하    | MPEG                             | <p>ISO 및 IEC 산하에서 비디오와 오디오 등 멀티미디어의 표준의 개발을 담당하는 소규모 그룹</p> <p>MPEG-H Part1 MMT(MPEG Media Transport) :E/D/S Layer<br/>MPEG-H Part2 HEVC<br/>MPEG-H Part3 3D AUDIO<br/>MPEG-21: 미래의 표준을 멀티미디어 프레임워크</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |                                                                                                                           |                       |
| 31 |          | 중    | MPEG-H                           | <p>UHDTV 콘텐츠 압축/전송 표준</p> <p>Part1: MMT(MPEG Media Transport)<br/>Part2: HEVC(High-Efficiency Video Coding)<br/>Part3: 3D Audio</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | 메비오                                                                                                                       |                       |

|    |           |      |                                       |                                                                                                                                                                                                                                                                                                                                                                                                                                               |                                                                                                |     |
|----|-----------|------|---------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|-----|
| 32 | 압축기술/영상처리 | 출제예상 | MPEG-I                                | [정의] 360 VR, MR 및 6-DoF 관련 콘텐츠를 포함한 비디오, 오디오, 시스템요소에 이르는 몰입형 미디어에 대한 MPEG 차세대 규격<br>[구성요소]<br>Part1 : Immersive Media architecture<br>Part2 : OMAF (Omnidirectional Media Format)<br>Part3 : Immersive Video<br>Part4 : Immersive Audio<br>Part5 : Point Cloud Compression<br>Part6 : Immersive Media Metrics<br>Part7 : Immersive Media Metadata                                                                                             | ISO/IEC 23090, OMAF, 3DoF, 3DoF+, 6DoF<br>Yaw/Roll/Pitch, Movement<br>Point cloud, Light field |     |
| 33 |           | 하    | 동영상 압축 기술                             | H.26x, HEVC                                                                                                                                                                                                                                                                                                                                                                                                                                   |                                                                                                |     |
| 34 |           | 중    | HEVC                                  | [정의] ISO/IEC 23008-2 표준 기반의 8K*4K 까지의 고해상도 및 고화질의 영상 스트리밍 서비스를 지원하는 블록기반 하이브리드 비디오 부호화 기술 (고효율압축)<br>(요구사항) 비트율, 해상도, 컬러 공간과 샘플링율, 프레임율, 복잡도, 부호화 모드, 스케닝방법<br>(구성요소) 복호기(역양자화, 루프필터, 화면예측, 움직임보상), 변환(변환, 양자화), 예측(움직임예측), 인트로피 부호화(CABAC)<br>[기술요소]<br>계층적부호화구조 - CU(Coding Unit), PU(Prediction Unit), TU(Transform Unit)                                                                                                                  |                                                                                                |     |
| 35 |           | 하    | 플렌옵틱 영상처리(Plenoptic Image Processing) | 차원 공간 모든 점에서 모든 방향으로 빛이 입사된 정보를 분리, 획득, 처리, 가시화 하는 영상 기술                                                                                                                                                                                                                                                                                                                                                                                      |                                                                                                |     |
| 36 | 최적해 알고리즘  | 중    | 탐욕 알고리즘                               | [정의] 결정해야 할 때마다 미래에 대한 생각없이 그 순간에 가장 최선의 선택을 하는 알고리즘<br>[특징] 순간 최적(locally optimal), 최종적 최적 보장 안함(no globally optimal), 항상 최적의 해를 주는지 검증 필요<br>[절차] 선정과정 - 적정성 점검 - 해당 점검<br><br>그리디 알고리즘의 핵심은 여러 선택에 있어서 앞으로의 선택을 고려하지 않고 각각의 선택마다 가장 최선의 선택을 하는 알고리즘. 즉, 우리가 선택할 경우에는 가장 최적의 선택만을 하지만 그리디 알고리즘은 각각 선택에 있어서는 최적일 뿐이지는 모르나 전체적으로 최적일 아닐 수도 있다는 것을 명심해야 한다.<br>주소: 근사적인 방법, 탐욕스런 선택 조건(greedy choice property)과 최적 부분 구조(optimal substructure) | ITPE 특강 필수 수강<br>122회 기술                                                                       | 해적점 |
| 37 |           | 중    | Beam Search                           | [정의] 여러 경우 중 하나를 결정하지 않고 Beam Size(K개)만큼 가장 가능성이 높은 후보군으로 선택하여 Greedy A 알고리즘의 최적해 미보장 단점을 보완한 최적해 알고리즘<br>[절차] 문제 정의, 최초 K개의, 해 선택, 부분 해 확장, K개 해 선택, 최종 해 선택                                                                                                                                                                                                                                                                                 |                                                                                                |     |
| 38 |           | 하    | 동적계획법(DP)                             | [정의] 주어진 문제를 여러 개의 소문제로 분할하여 각 소문제의 해결안을 바탕으로 주어진 문제를 해결하는 기법<br>[특징] 상향식 접근 방법(밑에서 시작), 순환적 성질(작은문제에서 더 큰 문제 결과찾기), 최적성 원리<br>[절차] 최적해 도출위한 재귀 관계식 도출 - 작은 문제부터 해 도출, 테이블 저장 - 점차적 상위 문제 해 도출<br>[원리] 재귀식도출 -> Memoization -> Bottom-up Approach<br>[조건] 최적 부분 구조(큰 문제의 최적 솔루션에 작은 문제 최적 솔루션 포함), 재귀<br>호출 시 중복(재귀적 해법으로 풀면 같은 문제에 대한 재귀호출이 심하게 중복)<br>[장점] 모든 해 고려해 항상 최적의 결과                                                               | 메모이제이션<br>상향식<br>O(n^2)<br>점화식<br>재귀식                                                          |     |
| 39 | 알고리즘      | 하    | 백트래킹(Backtracking)                    | [정의] 특정 조건을 충족시키는 후보해(가능성)을 모두 포함하는 모든해로부터 진짜 해찾기<br>[특징] 유망성(유망노드의 child만 검색), 깊이우선 검색<br>[절차] 상태공간트리 깊이 우선 검색 실시, 각 마디 유망(promising) 여부 점검, 유망하지 않을 시 부모마디로 돌아가서 검색 수행<br>[사례] maze problem, n-queen problem(서로 상대방 위협하지 않도록 n*n 서양 장기 판에 위치시킨, 같은 행/열/대각선 상 위치하지 않아야 함)<br>[사례] 1)미로 탈출                                                                                                                                                    | 122회 기술                                                                                        |     |
| 40 |           | 상    | 대기행렬이론 (Queueing Theory)              | [정의] 고객과 서비스 시설과의 관계를 확률론을 적용하여 운영역량 및 고객의 도착상황에 대응할 수 있는 경제적 규모를 결정하고자 하는 의사결정기법<br>[구성요소] 모집단 ( Calling population ), 대기행렬규칙 ( Queue discipline ), 진입포기 ( Balk ), 중도이탈 ( Reneging ), 줄바꿈 ( Jockeying )<br>[유형] 단일경로-단일과정, 복수경로-단일과정, 단일경로-복수과정, 복수경로-복수과정<br>[서비스선리규칙] Static Queue (FIFO, LIFO), Dynamic Queue (SRP)                                                                                                                       | Kanban<br>CPU 스케줄링                                                                             |     |
| 41 |           | 하    | 피보나치 수                                | void fibonacci(int n) {<br>int a = int[n];<br>a[0]=0; a[1]=1;<br>for(int i=2;i<=n;i++) {<br>a[i] = a[i-1] + a[i-2];<br>}<br>return a[n];<br>}                                                                                                                                                                                                                                                                                                 |                                                                                                |     |
| 42 | 기타        | 하    | 모라벡의 역설 (Moravec's Paradox)           | 인간은 일상으로 듣고 보고 느끼는 일은 매우 쉽게 할 수 있는 반면에 복잡한 수식 계산에서는 뒤처지고, 컴퓨터는 듣고 보고 느끼는 일은 매우 어렵지만 인간이 감당해낼 수 없는 상상을 초월하는 수식의 계산은 순식간에 해낸다는 주장. 모라벡 역설<br>예) 이중 보행, 음성인식, 위치 감지능력                                                                                                                                                                                                                                                                            |                                                                                                |     |